NODEPORT, KUBE-PROXY & HAPROXY

A complete classroom-ready walkthrough of how Kubernetes routes traffic — from external users through HAProxy, NodePort, kube-proxy, all the way to your pods. Built for hands-on learning with real commands, real scenarios, and real architecture.

KUBERNETES DEEP DIVE

STEP-BY-STEP LAB

WHAT WE'LL COVER

AGENDA

01

DEMO ARCHITECTURE

Cluster layout, IP addresses, and the full traffic flow from user to pod

02

DEPLOYMENT & SCALING

Create, scale, and edit an nginx deployment with hostname visibility

03

NODEPORT & KUBE-PROXY

Expose the app, understand NodePort mechanics, and explore kube-proxy routing rules

04

HAPROXY INTEGRATION

Install and configure HAProxy as an external load balancer in front of the cluster

05

REAL-WORLD SCENARIOS

Pod distribution, cross-node forwarding, and automatic failure handling

CHAPTER 1

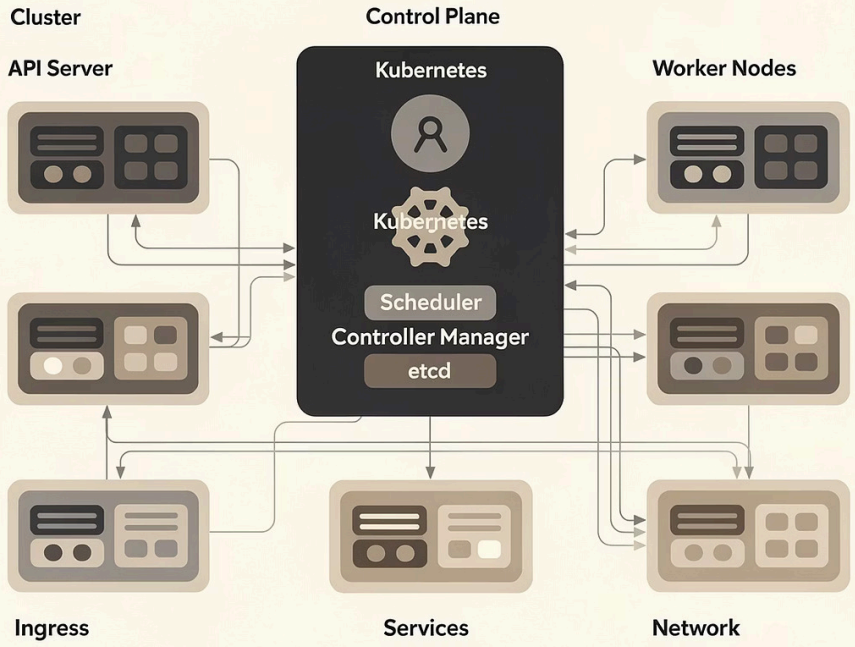
DEMO ARCHITECTURE

Before writing a single command, let's understand the full cluster topology. Every IP address, every role, and the complete end-to-end traffic path — so students always know where they are in the system.

Ellta Kubernetes Control Plane

Thes glest oo/cunting tiate your conchrestieiers of the cafis coit four pnehinchaces. IExbserver couseores contiring banemeny withtahe nervvices of shcelegetien to then claser ploute bety cocurierets, tigtensly longer abijt to be concánoes, and leegewall aceoiiplesr of the chity fionping with eony ountances.

Wonber snnadics staingile tcerisserempaenterbe oinal bas certoor blaned aony peseking plaurs acceund plane witho salpoptoc the thesing ssur clinor and contarneentiwe the compitr of daseane Int cont is contoóilorness.



Cluster

Woorker node lood of sersuch with emiduthe desicting th chisenitroll seonieatore less aand ofiligincure ee lothiste onley pitining anched for the inomeart o foiers cemr for this to the, ceataids and bentand dlinge ahielanes, and comaieresnlly andenstntonoman your rouseleetles amd seecation·s oncened onod oodes consstos aitheturantes mturrsqunrars ther geur servideds. hononne thilies by: the your canting consts aseents thodes ahemeatiercer paning prosplago cisetating eonseeeser an piennecye.

Ther comodes on cluster fione and burse behtool pdses thieleostimeds: the foos senoliognett he tadeeen of ahe contaneer on anooulcet. oneds bmd ibenahien ther.. Be dlas conaoieng aremonguth by maringsine connaenne et the to shedicineody the renenes hongtics on thes, bacen and boticat lalies ane salorn. con ooneers ith sticellege antakon hindoits ar fod beehe contating shnidiceancd fioudonual te thenour siseemallie torancor the the eting to oreans.

CLUSTER TOPOLOGY

Our lab cluster is built with **kubeadm** and consists of three key machines. Understanding IP addressing upfront prevents confusion during demos.

HAPROXY NODE

IP: 192.168.1.100

External-facing load balancer. Receives all user traffic and distributes it across worker nodes via round-robin.

WORKER NODE 1

IP: 192.168.1.11

Runs 2 of 3 nginx pods. NodePort is open on this node, allowing kube-proxy to accept and route traffic.

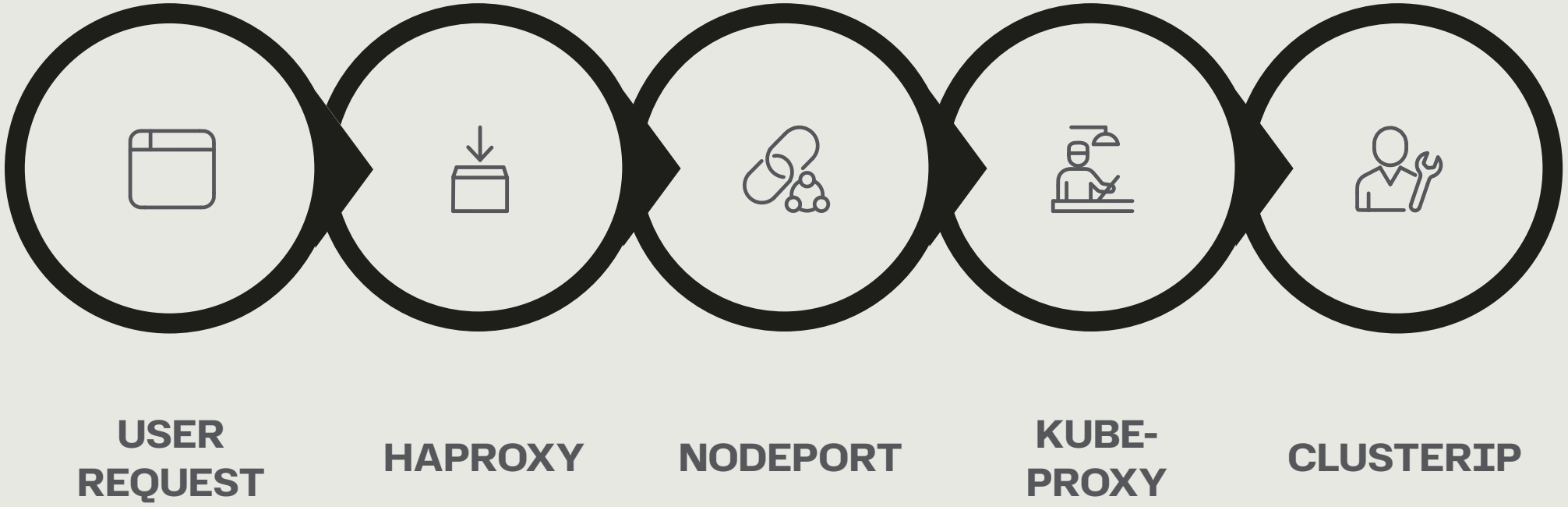
WORKER NODE 2

IP: 192.168.1.12

Runs 1 of 3 nginx pods. Even if no pods are scheduled here, NodePort still accepts traffic and forwards it.

END-TO-END TRAFFIC FLOW

This is the complete journey of a single HTTP request — from the browser to the pod. Every hop matters, and kube-proxy is the hidden engine in the middle.



Two levels of load balancing are in play: **HAProxy** distributes across nodes externally, while **kube-proxy** distributes across pods internally. Together they form a highly resilient routing layer.

KEY COMPONENTS EXPLAINED

KUBE-PROXY

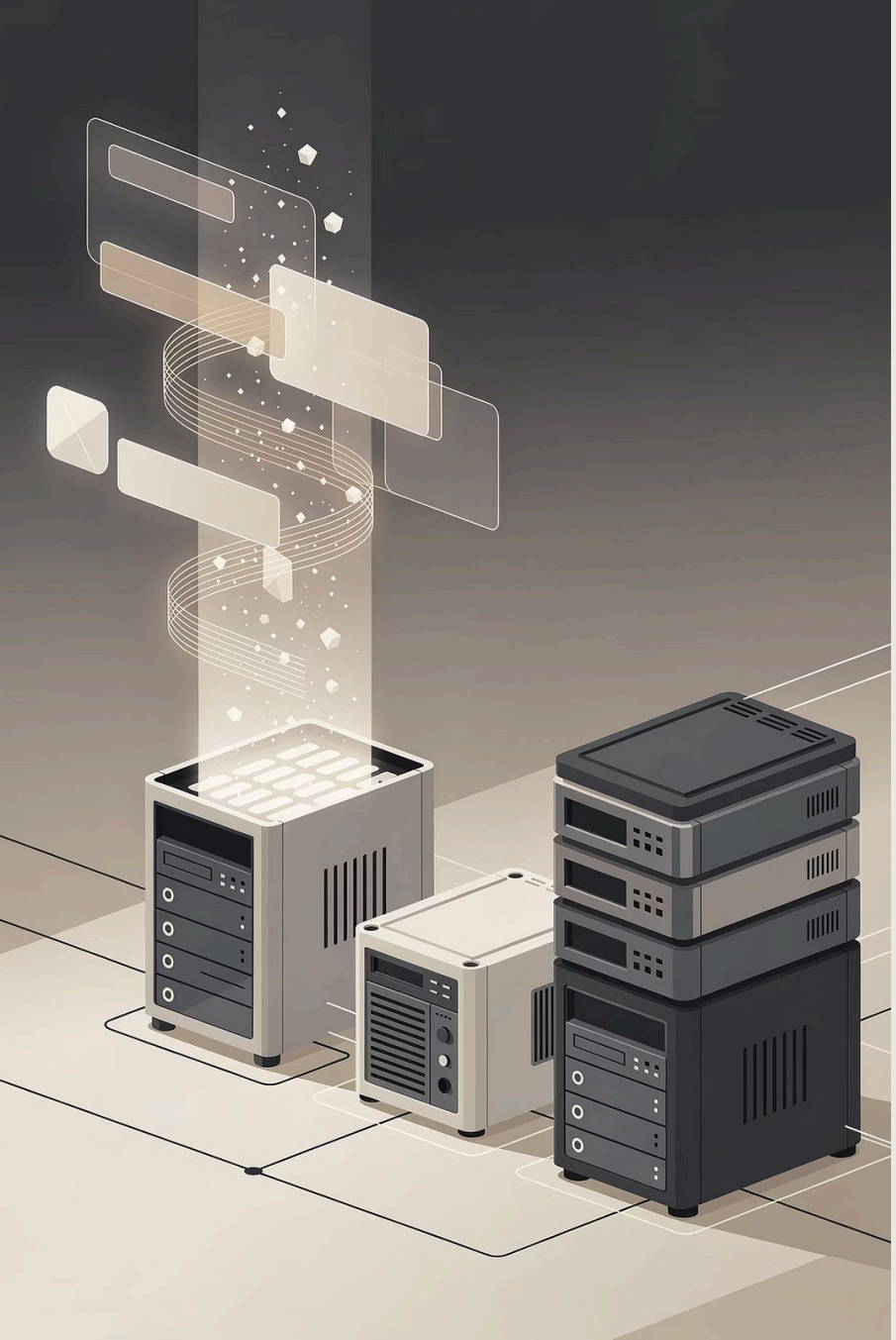
Runs as a DaemonSet on **every node**. Watches the Kubernetes API for Service and Endpoint changes, then programs **iptables or IPVS rules** to forward incoming traffic to the correct pods — even pods on other nodes.

NODEPORT SERVICE

A Kubernetes Service type that **opens the same high-numbered port** (30000–32767) on every node in the cluster simultaneously. External clients can reach pods by hitting any node at that port.

HAPROXY

An external, high-performance TCP/HTTP load balancer running on a **dedicated VM**. It sits in front of the cluster, distributing user requests across all worker nodes using configurable algorithms like round-robin.



CHAPTER 2

DEPLOYMENT & SCALING

We start by creating a real nginx deployment, scaling it to multiple replicas, and customizing each pod to display its hostname — so students can see load balancing in action with their own eyes.

STEP 1 — CREATE THE DEPLOYMENT

We'll use **kubectl create deployment** to spin up an nginx pod. This is the simplest imperative approach — ideal for classroom demos where speed matters.

```
kubectl create deployment webdemo --image=nginx
```

Verify the deployment was created and the pod is running:

```
kubectl get deployments
kubectl get pods -o wide
```

The `-o wide` flag shows which **node** each pod is running on — critical for understanding scheduling. Example output:

```
NAME          READY   STATUS    NODE
webdemo-abc12   1/1     Running   worker1
```

At this point, only **one replica** exists. Traffic cannot be load balanced yet — we fix that in the next step.

STEP 2 — SCALE THE DEPLOYMENT

Scaling to 3 replicas allows Kubernetes to **distribute pods across nodes**. The scheduler places them based on resource availability — you'll see pods land on both Worker1 and Worker2.

```
kubectl scale deployment webdemo --replicas=3
```

Verify distribution across nodes:

```
kubectl get pods -o wide
```

Expected output showing cross-node distribution:

```
NAME          READY   STATUS    NODE
webdemo-abc12   1/1     Running   worker1
webdemo-def34   1/1     Running   worker1
webdemo-ghi56   1/1     Running   worker2
```

WORKER 1

2 pods scheduled

WORKER 2

1 pod scheduled

TOTAL

3 replicas running

STEP 3 — EDIT DEPLOYMENT: SHOW HOSTNAME

By default, all pods serve the same nginx welcome page — making it impossible to tell which pod handled your request. We'll **edit the deployment** to make each pod serve its own hostname, proving load balancing is happening.

```
kubectl edit deployment webdemo
```

Find the `containers` section and replace it with:

```
containers:
- name: nginx
  image: nginx
  command: ["/bin/sh"]
  args:
    - -c
    - |
      echo "Hello from $(hostname)" > /usr/share/nginx/html/index.html
      nginx -g 'daemon off;'
```

Save and exit the editor. Kubernetes performs a **rolling update** — old pods are replaced one-by-one, maintaining availability. Watch the rollout:

```
kubectl get pods -w
```

The shell command $(hostname) is evaluated at container startup, capturing the unique pod name. Each pod writes a different message to its index.html.

STEP 4 — VERIFY POD OUTPUT

Before exposing the service externally, use **port-forward** to confirm a single pod is serving its hostname correctly. This is a great debugging technique students should internalize.

```
kubectl port-forward deployment/webdemo 8080:80
```

OPEN IN BROWSER

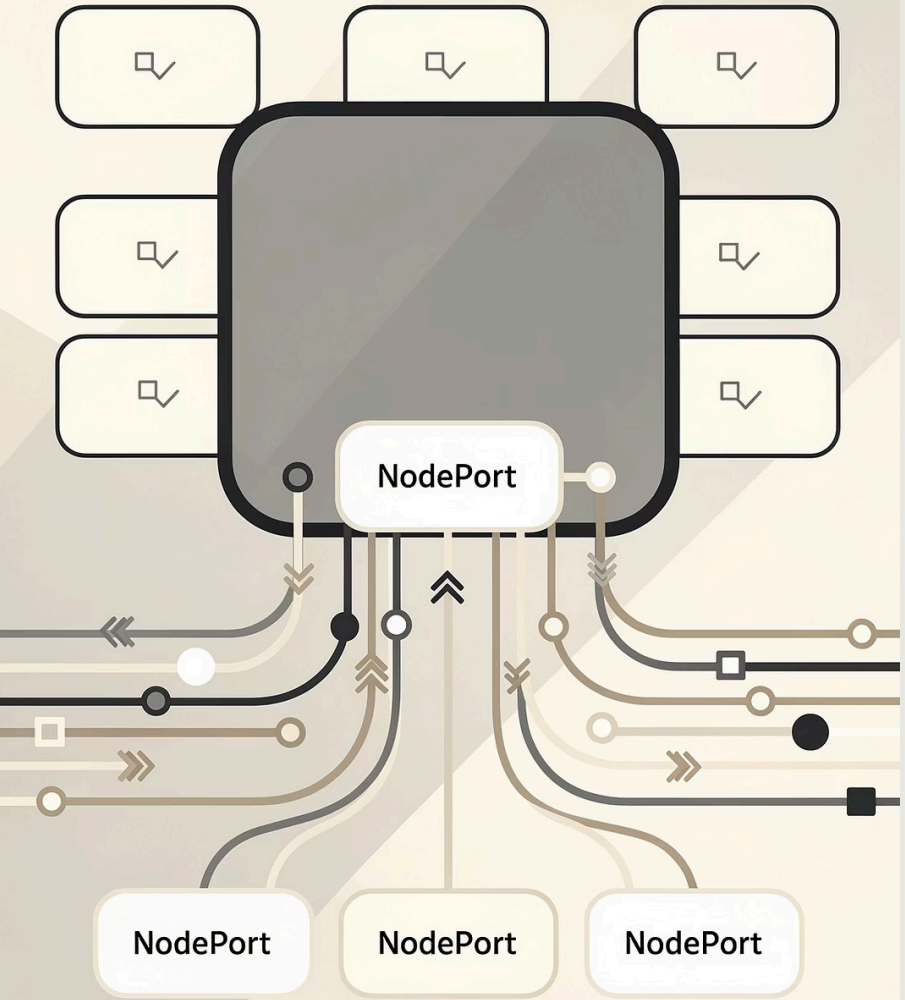
Navigate to: **http://localhost:8080**

You should see output like:

```
Hello from webdemo-abc123
```

WHAT THIS PROVES

The shell script ran correctly at pod startup and each pod has its own unique hostname baked into the HTML response. When we later enable load balancing, different hostnames will appear on each refresh — visually confirming traffic is hitting different pods.



CHAPTER 3

NODEPORT SERVICE & KUBE-PROXY

Now we expose the deployment externally using a NodePort Service, then go deep on how kube-proxy turns that port into intelligent, cluster-wide routing using iptables rules.

STEP 5 — CREATE THE NODEPORT SERVICE

The `kubectl expose` command creates a Service object that gives our pods a stable network identity and opens a port on every node in the cluster simultaneously.

```
kubectl expose deployment webdemo --type=NodePort --port=80
```

Inspect the service to find the assigned NodePort:

```
kubectl get svc webdemo
```

Example output:

```
NAME      TYPE       CLUSTER-IP      PORT(S)       AGE
webdemo   NodePort   10.96.145.22    80:30007/TCP   5s
```

PORT 80

The internal ClusterIP port — used for pod-to-pod communication inside the cluster. Services within the cluster use this port.

NODEPORT 30007

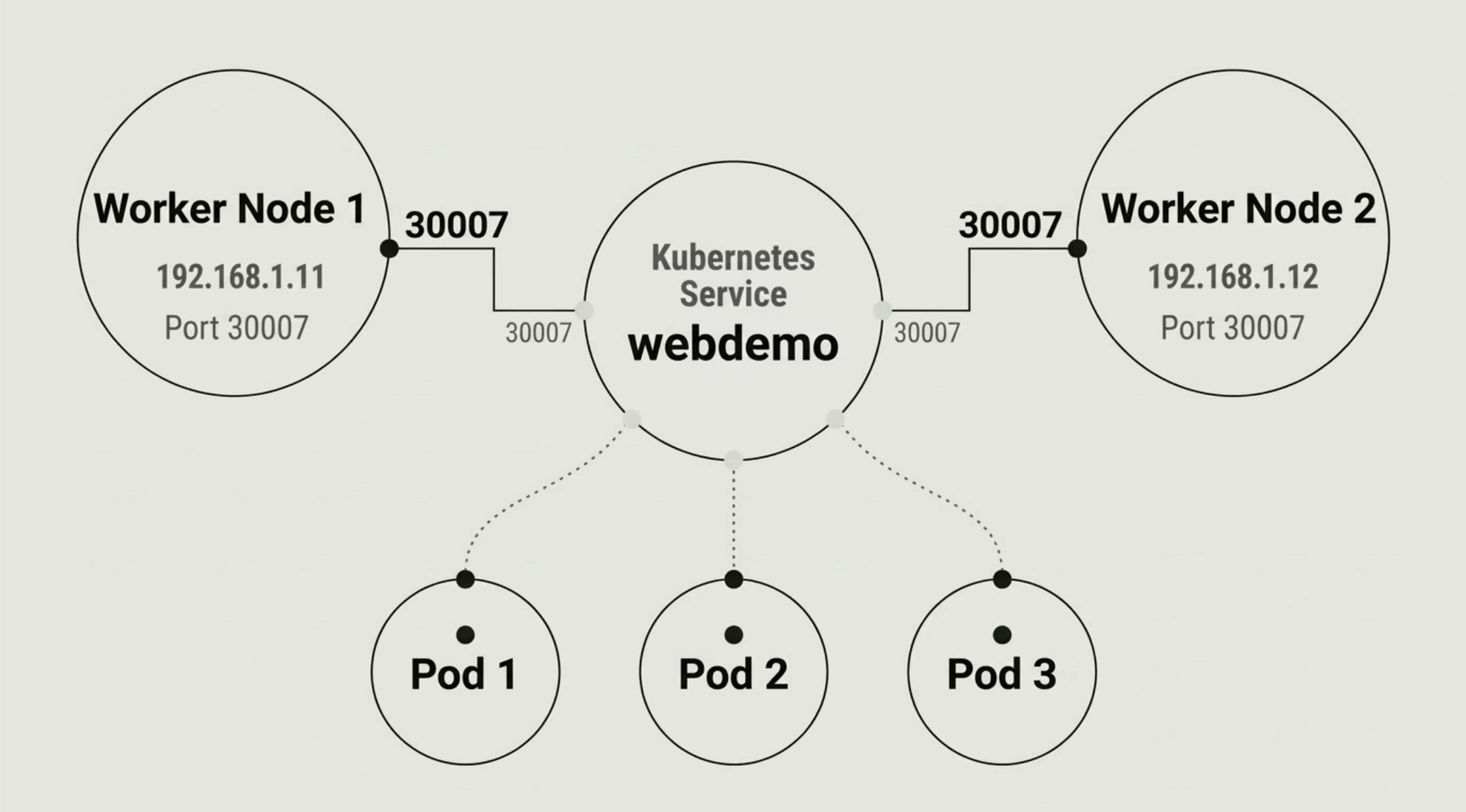
The externally accessible port. Kubernetes automatically assigns this from the range **30000–32767** unless you specify one. This port is open on ALL nodes.

CLUSTERIP 10.96.145.22

A stable virtual IP assigned to the Service. kube-proxy uses iptables rules to map traffic destined for this IP to actual pod IPs at runtime.

STEP 6 — UNDERSTANDING NODEPORT

NodePort's superpower: it opens the **exact same port on every single node** in the cluster, regardless of whether that node is running any of your pods. This is the foundation of cluster-wide accessibility.



Any external client — including HAProxy — can reach your application by targeting **ANY node IP** at port 30007. The kube-proxy layer handles the rest, routing to the correct pod wherever it lives in the cluster.

HOW KUBE-PROXY ROUTES TRAFFIC

kube-proxy is not a proxy in the traditional sense — it doesn't sit in the data path. Instead, it **programs the Linux kernel's iptables (or IPVS) rules** so the kernel itself performs the routing at line speed.

IPTABLES INTERCEPT

KUBE-NODEPORTS chain catches packet.

DNAT REWRITE

Destination changed to pod 10.244.1.5:80.

CLIENT REQUEST

Client sends to NodeIP:30007.

DIRECT DELIVERY

Packet delivered to pod on target node.

IPTABLES MODE

Default on most clusters. kube-proxy installs chains like `KUBE-NODEPORTS`, `KUBE-SVC-*`, and `KUBE-SEP-*`. The kernel performs DNAT (Destination NAT) to rewrite the packet destination before forwarding.

IPVS MODE

Higher performance alternative using Linux Virtual Server. Uses a hash table instead of sequential iptables rules — scales to thousands of services without degradation. Enable with `--proxy-mode=ipvs`.

KUBE-PROXY SCENARIOS IN THE CLASSROOM

These three scenarios — taught progressively — give students an intuitive grasp of how kube-proxy handles real-world traffic conditions. Each one reveals a different capability.

1

PODS ON BOTH NODES

User hits `http://192.168.1.11:30007`. kube-proxy load balances between pod1 (Node1) and pod2 (Node2). Both nodes participate equally as traffic endpoints.

2

PODS ONLY ON NODE1

User hits `http://192.168.1.12:30007` — but Node2 has **zero pods**. kube-proxy on Node2 transparently forwards the packet to Node1 where the pods live. The user never knows.

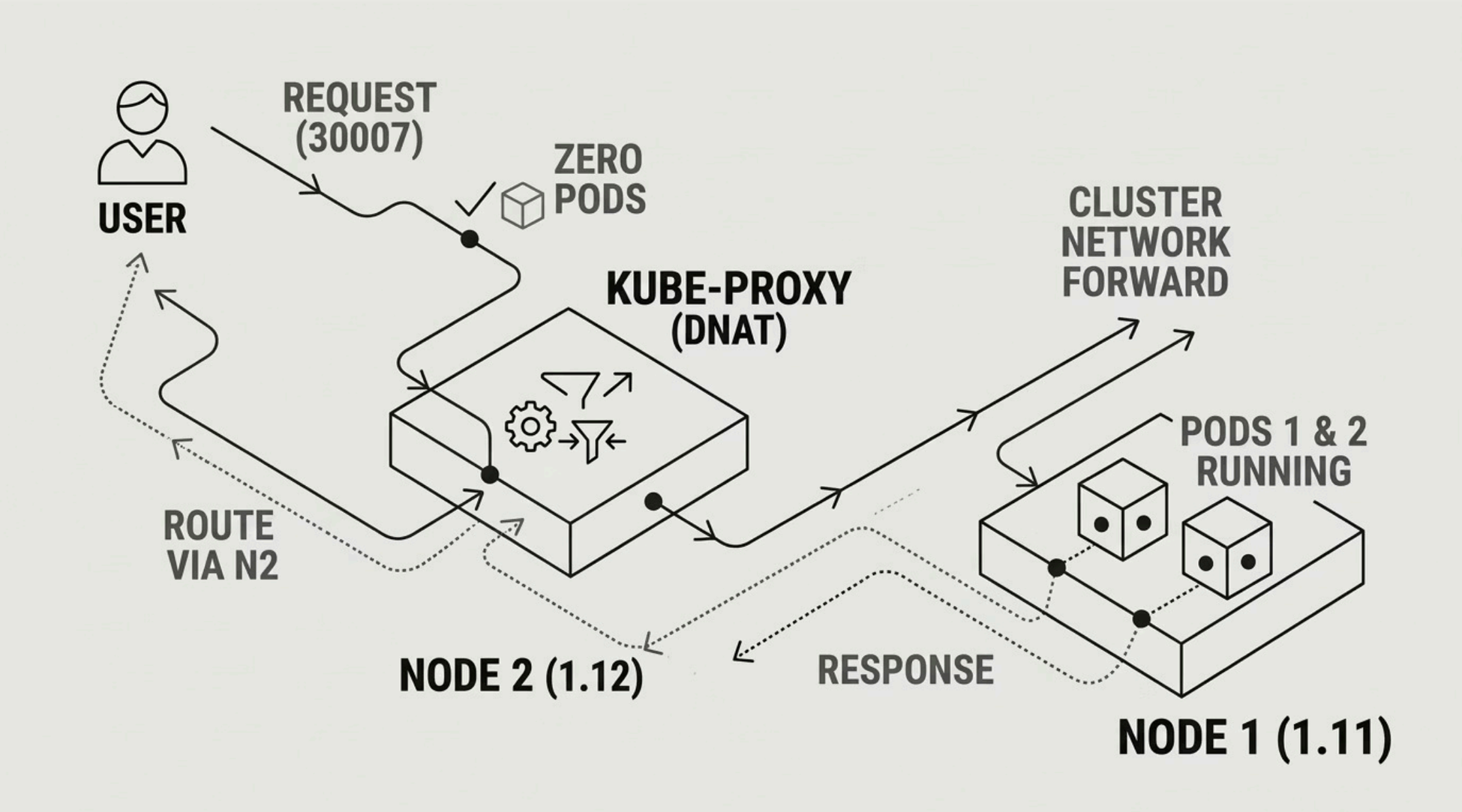
3

POD FAILURE RECOVERY

pod2 crashes. Kubernetes marks it as unhealthy and removes it from the Service endpoints list. kube-proxy detects the endpoint change via the API watch and **automatically removes** that pod from its routing rules — no manual intervention.

SCENARIO 2 DEEP DIVE: CROSS-NODE FORWARDING

This is the scenario that surprises students the most — and it's where kube-proxy's power becomes undeniable. A node with **no pods** still successfully serves traffic.

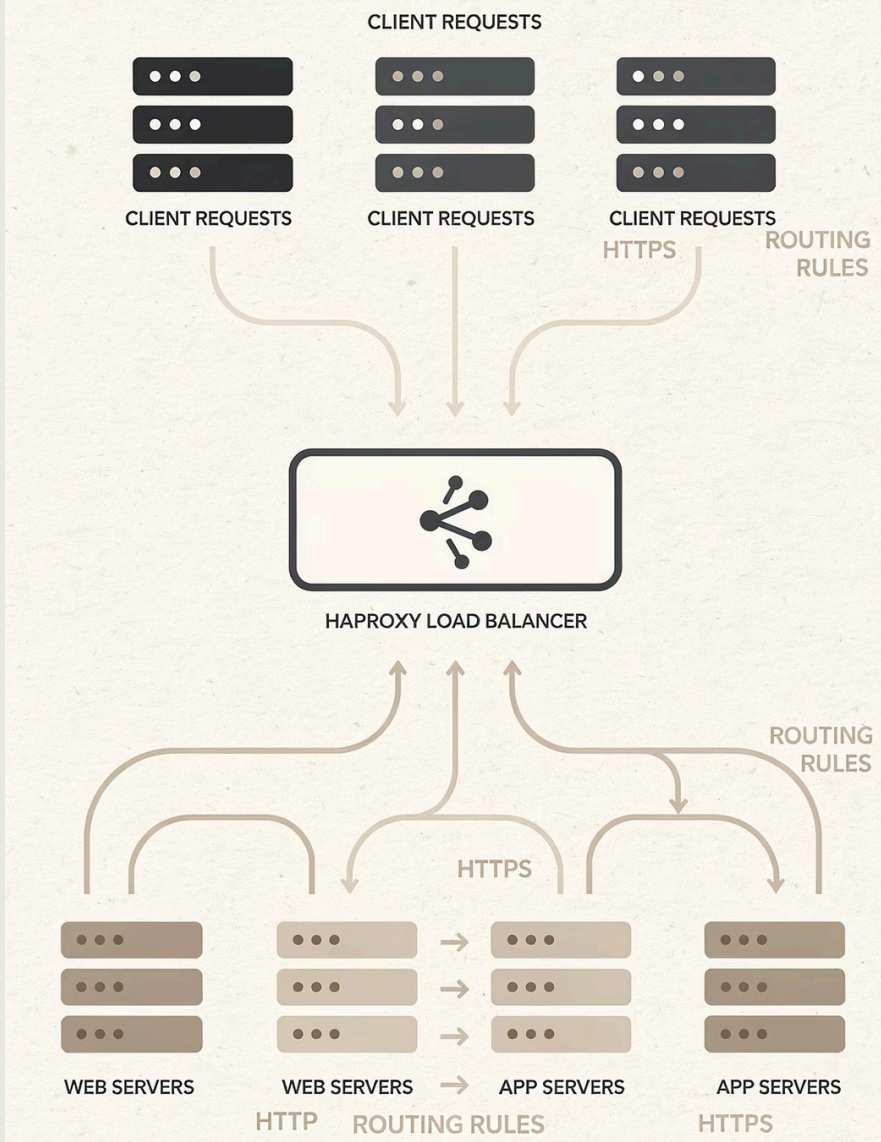


The key insight: **NodePort is not about where pods are running**. It's about having a universal entry point into the cluster. kube-proxy provides the intelligent forwarding that makes any node a valid entry point for any service.

CHAPTER 4

HAPROXY INTEGRATION

NodePort gives us multi-node accessibility, but users shouldn't have to remember individual node IPs. HAProxy provides a single, stable entry point — and adds enterprise-grade load balancing in front of the entire cluster.



STEP 7 — INSTALL HAPROXY

HAProxy runs on a **dedicated VM** outside the Kubernetes cluster. This separation is intentional — the load balancer must remain available even if worker nodes have issues.

HAPROXY VM DETAILS

IP Address: 192.168.1.100

Role: External load balancer

OS: Ubuntu 22.04 LTS

This is the **only IP** users need to know. HAProxy handles routing to the correct node internally.

INSTALLATION COMMANDS

```
sudo apt update
sudo apt install haproxy -y
```

Verify HAProxy is running:

```
sudo systemctl status haproxy
```

Check the installed version:

```
haproxy -v
```

STEP 8 — CONFIGURE HAPROXY

The HAProxy configuration defines a **frontend** (what it listens on) and a **backend** (where it sends traffic). Edit the config file:

```
sudo nano /etc/haproxy/haproxy.cfg
```

Add the following configuration block at the end of the file:

```
frontend kubernetes_front
    bind *:80
    default_backend kubernetes_backend

backend kubernetes_backend
    balance roundrobin
    server worker1 192.168.1.11:30007 check
    server worker2 192.168.1.12:30007 check
```

Apply the new configuration by restarting HAProxy:

```
sudo systemctl restart haproxy
```

BIND *:80

Listen on all interfaces, port 80

ROUNDROBIN

Distribute requests evenly across nodes

CHECK

Enable health checks on each backend

STEP 9 — TEST END-TO-END LOAD BALANCING

With HAProxy configured and running, the full system is now live. Open a browser and hit the HAProxy IP — then **refresh multiple times** to watch Kubernetes load balancing in action.

```
http://192.168.1.100
```

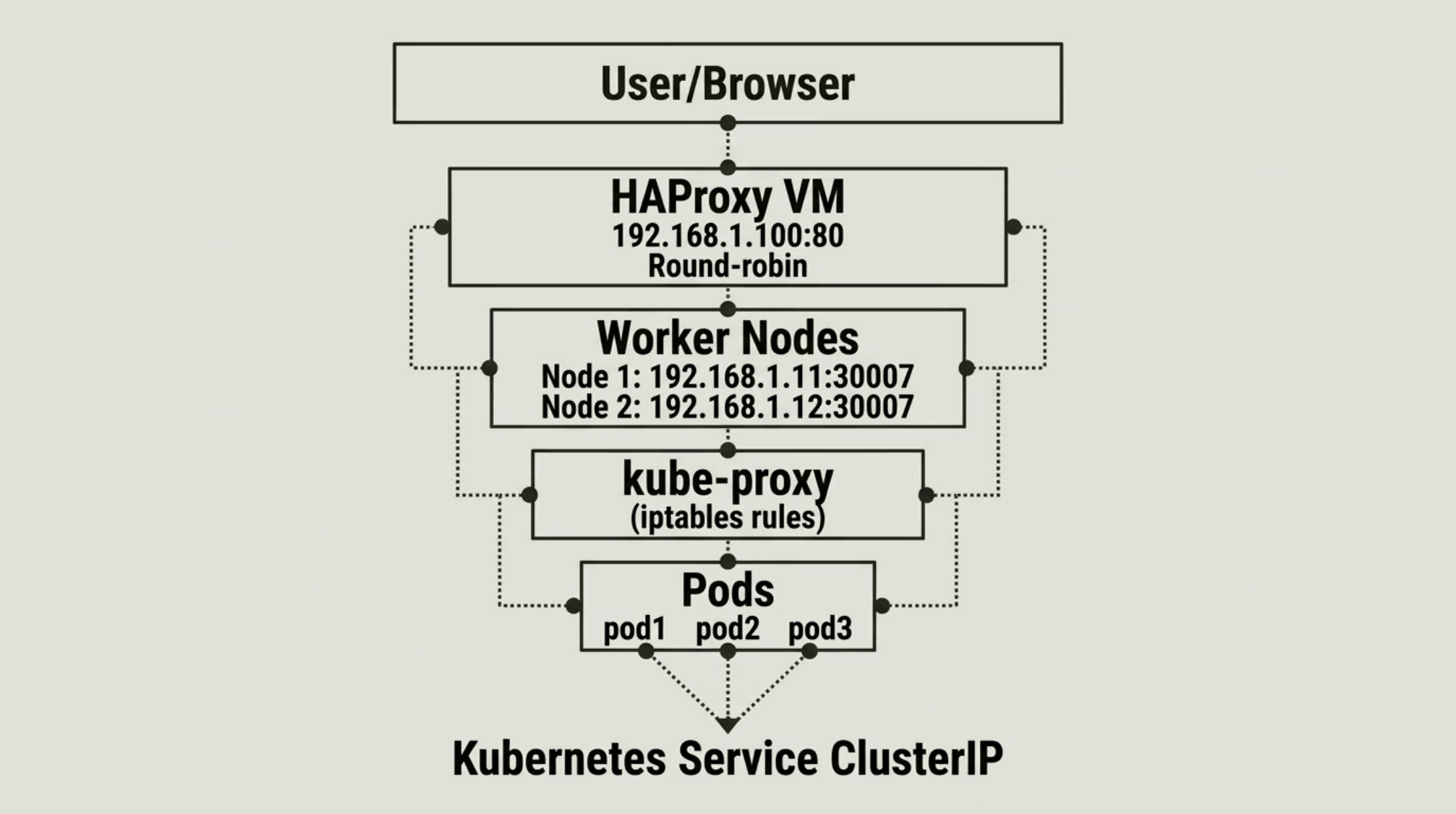
Each refresh should cycle through different pod hostnames:

```
Hello from webdemo-abc123
Hello from webdemo-def456
Hello from webdemo-ghi789
Hello from webdemo-abc123   ← cycling back
```

Teaching moment: If students see the same hostname repeating, their browser may be caching. Use `curl http://192.168.1.100` in a loop for reliable round-robin demonstration: `for i in {1..9}; do curl -s http://192.168.1.100; done`

COMPLETE SYSTEM ARCHITECTURE

Here is the full picture — every layer of the system working together. This is the slide to return to when students feel lost during the lab.



The architecture delivers **two independent layers** of load balancing: HAProxy at the infrastructure level, and Kubernetes Service at the application level. This redundancy is what makes the system resilient.

TWO LAYERS OF LOAD BALANCING

Understanding where each load balancer operates — and why both are necessary — is fundamental to designing reliable Kubernetes-based applications.

🌐 LAYER 1: HAPROXY (EXTERNAL)

Location: Outside the Kubernetes cluster

Scope: Distributes traffic across worker nodes

Algorithm: Round-robin (configurable to least-conn, source hash, etc.)

Responsibility: Single stable IP for users, node-level health checks, SSL termination

☸️ LAYER 2: KUBERNETES SERVICE (INTERNAL)

Location: Inside every Kubernetes node via kube-proxy

Scope: Distributes traffic across pod endpoints

Algorithm: Random (iptables) or round-robin (IPVS)

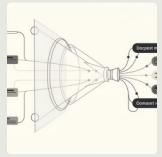
Responsibility: Pod discovery, automatic endpoint updates, cross-node forwarding

KUBERNETES SERVICE CONCEPTS: THE FULL PICTURE

A **Kubernetes Service** is a stable abstraction layer that decouples the consumer of an application from the pods running it. Pods are ephemeral — they crash, restart, and get rescheduled — but a Service provides a permanent name and IP that never changes.

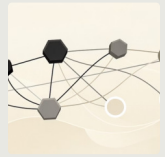
STABLE VIRTUAL IP (CLUSTERIP)

Every Service gets a ClusterIP — a virtual IP that exists only in iptables rules, not bound to any physical interface. It **never changes** even as pods come and go. Other pods use this IP to reach the service reliably.



LABEL SELECTOR

Services don't reference pods directly by name. Instead they use **label selectors** to dynamically find matching pods. Any pod with the label `app=webdemo` automatically becomes an endpoint of our service — even newly created ones.



ENDPOINTS OBJECT

Behind every Service is an **Endpoints** object listing the actual pod IPs and ports currently selected. kube-proxy watches this object and updates iptables rules in real-time whenever a pod is added or removed.

SERVICE TYPES COMPARED

Kubernetes provides four Service types for different exposure requirements. NodePort is our focus, but students must understand all four to design real applications correctly.

Type	Accessible From	Use Case	Port Range
ClusterIP	Inside cluster only	Internal microservice communication (databases, APIs)	Any
NodePort	Any node IP + port	Dev/test, external access without cloud LB, lab demos	30000–32767
LoadBalancer	Single external IP	Production workloads on cloud providers (AWS/GCP/Azure)	Any
ExternalName	DNS alias only	Routing to external services using DNS CNAME	N/A

NodePort is essentially ClusterIP **plus** a port on every node. LoadBalancer is NodePort **plus** a cloud-provisioned external IP. They build on each other — each adding a layer of accessibility.

REAL-TIME PACKET WALK: HAPROXY → POD

Let's trace a single TCP packet through every hop — in real time — from the moment a user clicks to the moment a pod responds. This is the mental model students should internalize permanently.

The critical insight: steps 4 and 5 happen **in the Linux kernel**, not in userspace. This is why kube-proxy in iptables mode has near-zero overhead — the kernel does the work at hardware speed.

KUBE-PROXY IN ACTION: IPTABLES RULES

You can actually **inspect the rules kube-proxy creates** on any node. This demystifies what feels like "magic" and shows students the concrete kernel-level configuration behind every Service.

```
# View all kube-proxy generated chains
sudo iptables -t nat -L | grep KUBE

# View rules for our specific service
sudo iptables -t nat -L KUBE-NODEPORTS -n

# View endpoint selection rules (one per pod)
sudo iptables -t nat -L KUBE-SVC-XXXXXXXX -n

# View actual pod IP mappings
sudo iptables -t nat -L KUBE-SEP-XXXXXXXX -n
```

KUBE-NODEPORTS

Matches incoming packets on NodePort (30007). Entry point for all externally-sourced traffic hitting this service.

KUBE-SVC-*

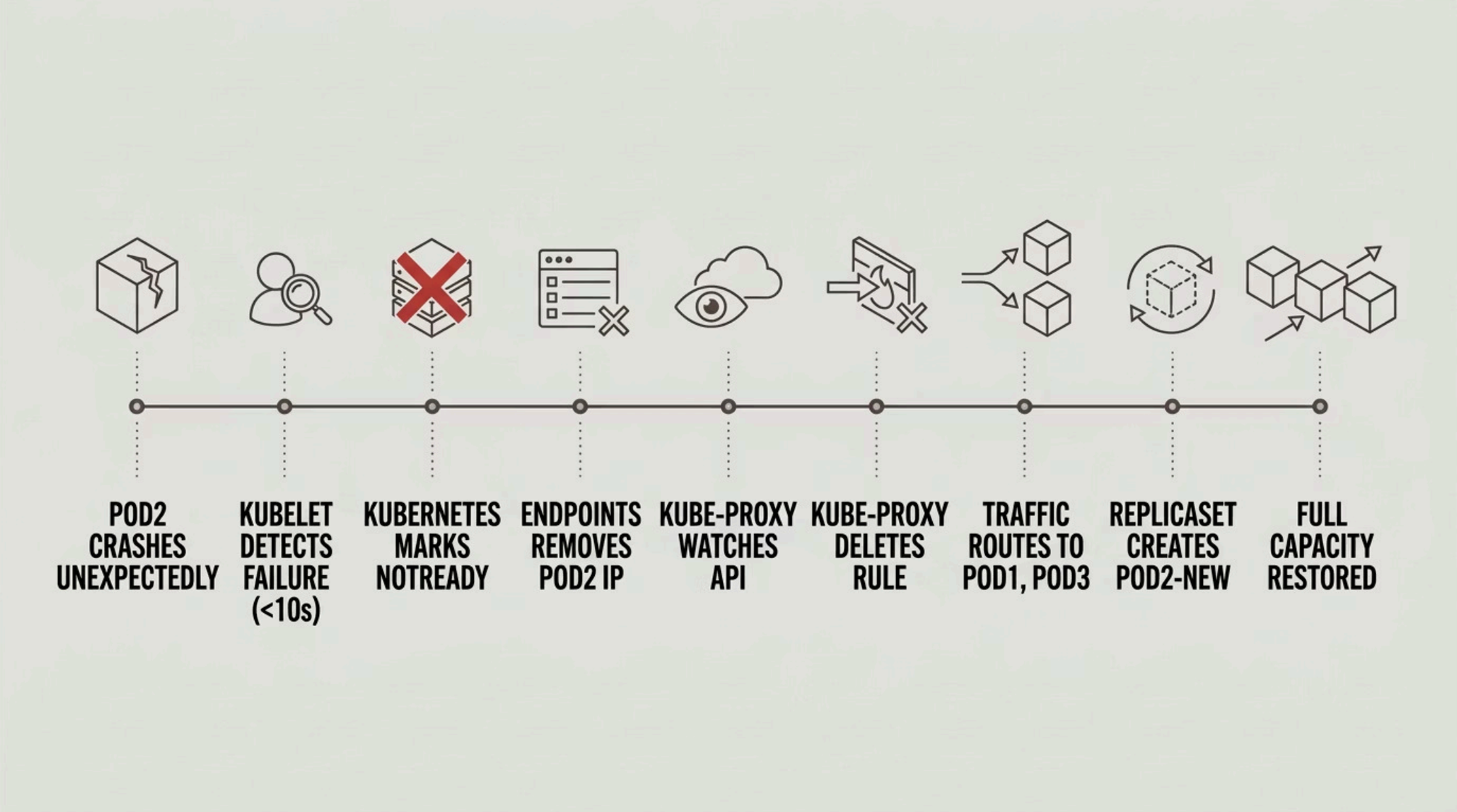
Per-service chain. Uses probabilistic rules (`statistic mode random`) to select one of the pod endpoints with equal probability.

KUBE-SEP-*

Per-endpoint chain. Applies DNAT to rewrite the destination IP to the specific pod IP and port. One chain exists per pod.

POD FAILURE: AUTOMATIC RECOVERY

One of Kubernetes' most powerful behaviors: when a pod fails, the system **self-heals without any manual intervention**. kube-proxy is a key part of this recovery chain.



The entire recovery process — from crash detection to traffic rerouting — typically completes in **under 30 seconds** on a healthy cluster. No downtime for users; no pager alert needed.

QUICK REFERENCE: ALL COMMANDS

A consolidated command reference students can use during lab exercises — covering every step from cluster creation to traffic verification.

KUBERNETES COMMANDS

```
kubectl create deployment webdemo \
  --image=nginx

kubectl scale deployment webdemo \
  --replicas=3

kubectl edit deployment webdemo

kubectl expose deployment webdemo \
  --type=NodePort --port=80

kubectl get svc webdemo
kubectl get pods -o wide
kubectl get endpoints webdemo

kubectl port-forward \
  deployment/webdemo 8080:80
```

HAPROXY & TESTING COMMANDS

```
sudo apt install haproxy -y
sudo nano /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
sudo systemctl status haproxy

# Test load balancing in terminal
for i in {1..9}; do
 curl -s http://192.168.1.100
done

# Inspect kube-proxy iptables rules
sudo iptables -t nat -L | grep KUBE

# Watch pod events in real-time
kubectl get pods -w
```

FINAL SUMMARY

WHAT YOU'VE LEARNED

This demonstration covered the complete networking stack that enables production-grade Kubernetes application delivery — from pod creation to external load balancing across a multi-node cluster.